AcciSure

Accidental Insurance Management Platform

Project Documentation Report

Complete Technical & Functional Reference

Table of Contents

Table of Contents 2
1. Project Title 4
2. Abstract 4
3. Problem Statement 5
4. Proposed Solution 6
4.1 Automated Digital Onboarding 6
4.2 Intelligent Policy Discovery with AI Chatbot 6
4.3 Smart Agent Assignment 6
4.4 Real-Time Communication — Chat and Video Calls 6
4.5 AI-Assisted Claims Processing 6
4.6 Dynamic Coverage Deduction 6
4.7 Automated Email Notifications 6
5. Key Innovations 7
6. Platform Roles 8
6.1 Customer 8
6.2 Agent 8
6.3 Claims Officer 9
6.4 Admin 9
7. Database Schema — Table Structures 10
7.1 AspNetRoles 10
7.2 AspNetUsers 10
7.3 AspNetUserRoles 11
7.4 AspNetUserClaims 11
7.5 AspNetUserLogins 12
7.6 AspNetUserTokens 12
7.7 AspNetRoleClaims 12
7.8 Notifications 12
7.9 PolicyApplications 13
7.10 InsuranceClaims 14
7.11 ClaimDocuments 15
7.12 Chats 16
7.13 ChatMessages 16
7.14 Database Relationship Summary 17
8. API Controllers and Endpoints 18
8.1 AuthController — /api/auth 18

8.2 AdminController — /api/admin 18
8.3 AgentController — /api/agent 19
8.4 PolicyController — /api/policy 19
8.5 ClaimController — /api/claim 20
8.6 ChatController — /api/chat 21
8.7 NotificationsController — /api/notifications 21
8.8 PaymentController — /api/payment 21
8.9 ReportController — /api/report 21
8.10 DashboardController — /api/dashboard 22
8.11 Error Handling — Standard Response Format 22
9. System Requirements 23
9.1 Frontend Development Requirements 23
9.2 Backend Development Requirements 24
10. System Architecture 25
10.1 Architecture Overview 25
10.2 Data Flow Architecture 25
10.3 Security Architecture 26
11. Workflow Summaries — Login to Logout 27
11.1 Customer Workflow 27
11.2 Agent Workflow 28
11.3 Claims Officer Workflow 28
11.4 Admin Workflow 29
12. Unique Selling Propositions (USPs) 30
13. Conclusion 31

1. Project Title

AcciSure

An End-to-End AI-Powered Accidental Insurance Management Platform

AcciSure is a full-stack, role-based digital insurance platform that streamlines the entire lifecycle of accidental insurance — from customer onboarding and policy selection to claims processing and settlement — using modern web technologies, real-time communication, AI-powered assistance, and automated workflows.

2. Abstract

The insurance industry has historically been plagued by manual, paper-heavy processes that result in delayed policy approvals, opaque claims handling, and poor customer experience. AcciSure addresses these challenges head-on by delivering a fully digital, role-driven accidental insurance ecosystem designed for the modern era.

AcciSure supports four distinct user roles — Customer, Agent, Claims Officer, and Admin — each operating within a precisely defined workflow. Customers can register with OTP and CAPTCHA verification, explore individual and family accidental insurance plans, interact with an AI-powered chatbot to understand policy details, and submit policy applications online. Agents review and approve or reject applications, manage their customer portfolio, and communicate with customers via real-time chat and video calls. Claims Officers review uploaded medical and incident documents enriched with AI-generated summaries and risk assessments, then approve or reject claims with defined payout amounts. The Admin oversees the entire system — creating staff accounts, assigning workloads intelligently based on current load, and monitoring KPI dashboards.

The platform is built with a RESTful API backend featuring JWT-based authentication, a SQL Server relational database, and integrates with external services including cloud document storage (ImageKit), AI workflow automation (N8N), OTP verification, and email automation. The result is a seamless, transparent, and highly efficient digital insurance experience.

3. Problem Statement

Traditional accidental insurance platforms suffer from several critical limitations that reduce efficiency and customer trust:

- Manual Policy Processing: Policy applications are processed through paperwork, phone calls, and in-person visits, causing delays of days or weeks.
- Lack of Transparency: Customers have no visibility into where their application or claim stands in the pipeline at any given time.
- Unbalanced Workload Assignment: Assigning agents and claims officers to cases is often done arbitrarily, creating bottlenecks with overloaded staff while others remain idle.
- Poor Communication Channels: Customers must call helplines or visit offices to clarify policy doubts or track claim status, creating friction and dissatisfaction.
- Inefficient Claims Review: Claims officers manually read through stacks of uploaded documents without any AI support, making the review process slow and error-prone.
- No Real-Time Interaction: There is no in-platform mechanism for customers and agents to communicate directly, leading to miscommunication and delays.
- Credential Management Complexity: Admin teams struggle to efficiently create and distribute staff credentials securely and promptly.

These problems together result in high operational costs, poor customer retention, delayed settlements, and increased fraud risk. AcciSure is built specifically to solve each of these problems through automation, AI, and real-time digital workflows.

4. Proposed Solution

AcciSure proposes a comprehensive, role-based digital platform with the following solution pillars:

4.1 Automated Digital Onboarding

Customers register through a secure process involving OTP (One-Time Password) mobile verification and CAPTCHA validation, ensuring authenticity. JWT tokens are issued on successful login for all subsequent authenticated requests.

4.2 Intelligent Policy Discovery with AI Chatbot

Customers can browse Individual and Family Accidental Insurance plans across Normal, Gold, and Premium tiers. An AI-powered chatbot (integrated via N8N workflow automation) opens in a sidebar when a customer clicks 'Interest to Buy', answering all policy-related questions before the customer commits to purchase.

4.3 Smart Agent Assignment

After a customer submits a policy application, the Admin reviews available agents and assigns the one with the least current workload, ensuring balanced distribution. This assignment is automated through real-time dashboard metrics.

4.4 Real-Time Communication — Chat and Video Calls

Once a policy is active, customers and their assigned agents can communicate in real-time via an in-platform messaging system similar to WhatsApp. Additionally, a video conferencing feature allows face-to-face interaction for complex discussions — a key USP of the platform.

4.5 AI-Assisted Claims Processing

When a customer raises a claim with supporting documents (FIR report, hospital report, accident report), these files are stored in cloud storage and their URLs saved in the database. The assigned Claims Officer's dashboard retrieves an AI-generated analysis via N8N including: a document summary, risk factor score, suggested claim amount range, and risk rating — significantly reducing review time and improving decision quality.

4.6 Dynamic Coverage Deduction

Approved claim amounts are automatically deducted from the policy's remaining coverage. For example, a policy with Rs.10 lakh coverage that has a Rs.2 lakh claim approved retains Rs.8 lakh in remaining coverage, with full auditability in the database.

4.7 Automated Email Notifications

Admin can dispatch credentials to newly created Agent and Claims Officer accounts via email automation, ensuring secure and prompt access provisioning.

5. Key Innovations

#	Innovation	Description
1	AI Policy Chatbot (N8N)	Embedded sidebar chatbot powered by N8N workflow AI answers customer queries about specific policies in real time before purchase.
2	AI Claims Analysis (N8N)	AI automatically extracts and analyzes claim documents to produce a summary, risk score, suggested payout range, and risk rating for the Claims Officer.
3	Video Call Integration	In-platform video conferencing between customers and agents enables face-to-face policy clarification — a unique feature in the insurance domain.
4	Real-Time Chat Messaging	WhatsApp-style messaging between customers and their dedicated agents, with read receipts and message history, all within the platform.
5	Smart Load-Based Assignment	Admin dashboard displays live workload metrics per agent and claims officer, enabling one-click assignment to the least-loaded staff member.
6	Dynamic Coverage Tracking	RemainingCoverageAmount is updated automatically in the database on every approved claim, providing real-time policy status without manual reconciliation.
7	Cloud Document Management	Claim documents are uploaded to cloud storage (ImageKit), and their secure URLs are stored in SQL Server, enabling scalable, reliable document retrieval.
8	10% Commission Automation	Agent commissions (10% of premium) are auto-calculated and recorded against each agent's bank account upon policy activation.
9	OTP + CAPTCHA Dual Verification	Two-factor registration security combining phone OTP and CAPTCHA ensures only genuine customers can create accounts.
10	Automated Credential Emails	Admin-triggered email automation instantly delivers login credentials to newly created Agent and Claims Officer accounts.

6. Platform Roles

AcciSure operates with four distinct roles, each with clearly defined responsibilities and access privileges.

6.1 Customer

The Customer is the primary end-user of the platform who purchases and manages accidental insurance policies.

- Register using email, phone (OTP), and CAPTCHA verification.
- Browse and explore Individual and Family Accidental Insurance plans.
- Interact with the AI chatbot to ask questions about specific policies before buying.
- Calculate premium quotes based on age, profession, habits, and travel frequency.
- Submit a formal policy application and track its status.
- Make premium payments after agent approval to activate the policy.
- Chat with the assigned agent via real-time messaging.
- Initiate video calls with the assigned agent for detailed discussions.
- Raise claims on active policies by providing incident details and uploading supporting documents.
- Track claim status and view remaining coverage after approved claims.
- Receive real-time notifications for all status changes.

6.2 Agent

The Agent acts as the intermediary between customers and the insurance company, responsible for reviewing applications and maintaining customer relationships.

- Receive assigned policy applications from the Admin.
- Review complete customer application details end-to-end.
- Accept or reject policy applications with documented decisions.
- View and manage their assigned customer portfolio.
- Communicate with customers via in-platform chat and video calls.
- Track commission earnings (10% of premium per activated policy).
- Access analytics including coverage portfolio, tier breakdown, monthly commission trends, and customer counts.
- View claims raised by their customers.
- Access unified payment reports for their portfolio.

6.3 Claims Officer

The Claims Officer is responsible for reviewing, evaluating, and deciding on insurance claims assigned by the Admin.

- Receive assigned claims from the Admin.
- Access the claims queue and view full claim details including incident information and supporting documents.
- Review AI-generated analysis: document summary, risk factor, suggested claim amount, and risk rating.
- Approve or reject claims with a specified payout amount and remarks.
- View all claims in their queue with their current statuses.
- Access claim payout reports for transactions they have handled.

6.4 Admin

The Admin has full system access and is responsible for user management, workload assignment, and platform oversight.

- Create Agent and Claims Officer accounts and distribute credentials via email automation.
- View all policy applications across all statuses.
- Assign policy applications to agents based on current workload metrics.
- View all insurance claims across the platform.
- Assign pending claims to Claims Officers based on their current load.
- Access the KPI dashboard with real-time statistics: total users, policies, claims, amounts.
- Manage all users (customers, agents, officers) including deletion.
- Access unified financial and payment reports across the entire platform.

7. Database Schema — Table Structures

The AcciSure platform uses Microsoft SQL Server as its primary relational database. The schema follows ASP.NET Core Identity conventions for authentication tables, extended with custom business tables. Below are all 13 tables with their exact column names, data types, and constraints as defined in the SQL schema.

7.1 AspNetRoles

Stores role definitions for the four platform roles: Admin, Agent, Customer, ClaimOfficer.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique role identifier (GUID)
Name	NVARCHAR(256)	NULL	Role name (e.g., 'Admin', 'Agent')
NormalizedName	NVARCHAR(256)	UNIQUE INDEX	Uppercase role name for lookups
ConcurrencyStamp	NVARCHAR(MAX)	NULL	Concurrency token

7.2 AspNetUsers

Central user table storing all platform users: Customers, Agents, Claims Officers, and Admins. Extended with custom fields for insurance-specific data.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique user identifier (GUID)
FullName	NVARCHAR(100)	NOT NULL	User's full name
AadhaarNumber	NVARCHAR(12)	NULL	12-digit Aadhaar for Customer KYC
BankAccountNumber	NVARCHAR(16)	NULL	16-digit bank account for Agent commissions
CreatedAt	DATETIME2	DEFAULT GETUTCDATE()	Account creation timestamp
InitialPassword	NVARCHAR(MAX)	NULL	Stored for Admin-created accounts (Agent/Officer)
UserName	NVARCHAR(256)	UNIQUE INDEX	Login username (same as email)
NormalizedUserName	NVARCHAR(256)	UNIQUE INDEX	Uppercase username for lookups
Email	NVARCHAR(256)	NULL	User email address
NormalizedEmail	NVARCHAR(256)	INDEX	Uppercase email for lookups
EmailConfirmed	BIT	DEFAULT 0	Email verification status

Column	Data Type	Constraints	Description
PasswordHash	NVARCHAR(MAX)	NULL	Hashed password (ASP.NET Identity)
SecurityStamp	NVARCHAR(MAX)	NULL	Security token for invalidating sessions
ConcurrencyStamp	NVARCHAR(MAX)	NULL	Optimistic concurrency token
PhoneNumber	NVARCHAR(MAX)	NULL	Phone number with country code
PhoneNumberConfirmed	BIT	DEFAULT 0	OTP verification status
TwoFactorEnabled	BIT	DEFAULT 0	Two-factor authentication flag
LockoutEnd	DATETIMEOFFSET	NULL	Account lockout expiry
LockoutEnabled	BIT	DEFAULT 0	Whether lockout is enabled
AccessFailedCount	INT	DEFAULT 0	Failed login attempt count

7.3 AspNetUserRoles

Junction table mapping users to their assigned roles. Enables many-to-many user-role relationships.

Column	Data Type	Constraints	Description
UserId	NVARCHAR(450)	PK, FK -> AspNetUsers(Id) CASCADE	Reference to user
RoleId	NVARCHAR(450)	PK, FK -> AspNetRoles(Id) CASCADE	Reference to role

7.4 AspNetUserClaims

Column	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-increment identifier
UserId	NVARCHAR(450)	FK -> AspNetUsers(Id) CASCADE	Reference to user
ClaimType	NVARCHAR(MAX)	NULL	Type of claim (e.g., role, permission)
ClaimValue	NVARCHAR(MAX)	NULL	Value of the claim

7.5 AspNetUserLogins

Column	Data Type	Constraints	Description
LoginProvider	NVARCHAR(450)	PK	External login provider name
ProviderKey	NVARCHAR(450)	PK	Unique key from external provider
ProviderDisplayName	NVARCHAR(MAX)	NULL	Display name of login provider
UserId	NVARCHAR(450)	FK -> AspNetUsers(Id) CASCADE	Reference to user

7.6 AspNetUserTokens

Column	Data Type	Constraints	Description
UserId	NVARCHAR(450)	PK, FK -> AspNetUsers(Id) CASCADE	Reference to user
LoginProvider	NVARCHAR(450)	PK	Token provider name
Name	NVARCHAR(450)	PK	Token name/type
Value	NVARCHAR(MAX)	NULL	Token value

7.7 AspNetRoleClaims

Column	Data Type	Constraints	Description
Id	INT	PRIMARY KEY, IDENTITY(1,1)	Auto-increment identifier
RoleId	NVARCHAR(450)	FK -> AspNetRoles(Id) CASCADE	Reference to role
ClaimType	NVARCHAR(MAX)	NULL	Type of role-level claim
ClaimValue	NVARCHAR(MAX)	NULL	Value of the role-level claim

7.8 Notifications

Stores all user-facing notifications triggered by system events such as policy status changes and claim updates.

Column	Data Type	Constraints	Description
Id	UNIQUEIDENTIFIER	PRIMARY KEY, DEFAULT NEWID()	Unique notification ID (GUID)
UserId	NVARCHAR(450)	FK -> AspNetUsers(Id) CASCADE	Target user

Column	Data Type	Constraints	Description
Title	NVARCHAR(200)	NOT NULL	Notification headline
Message	NVARCHAR(MAX)	NOT NULL	Full notification message body
CreatedAt	DATETIME2	DEFAULT GETUTCDATE()	Creation timestamp (UTC)
IsRead	BIT	DEFAULT 0	Whether notification has been read
NotificationType	NVARCHAR(50)	DEFAULT 'General'	Category: Policy, Claim, Payment, General

7.9 PolicyApplications

The central business table storing all insurance policy applications from submission through activation. Tracks the full lifecycle including coverage amounts, payments, and agent assignment.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique policy application ID
UserId	NVARCHAR(450)	FK -> AspNetUsers (Id) CASCADE	Customer who applied
PolicyCategory	NVARCHAR(MAX)	NOT NULL	e.g., 'Health Insurance - Individual'
TierId	NVARCHAR(MAX)	NOT NULL	GOLD or SILVER tier
CalculatedPremium	DECIMAL(18,2)	NOT NULL	Final calculated premium (INR)
SubmissionDate	DATETIME2	DEFAULT GETUTCDATE()	Date application was submitted
Status	NVARCHAR(MAX)	DEFAULT 'Pending'	Pending/Assigned/Approved/Rejected/Active/Expired
ApplicationDataJson	NVARCHAR(MAX)	NOT NULL	Full applicant data as JSON (name, age, profession, habits, nominee, members)
AssignedAgentId	NVARCHAR(450)	FK -> AspNetUsers (Id) NO ACTION	Agent assigned by Admin
ApprovedByAgentId	NVARCHAR(MAX)	NULL	ID of agent who approved
ApprovedAt	DATETIME2	NULL	Timestamp of agent approval

Column	Data Type	Constraints	Description
TotalCoverageAmount	DECIMAL(18,2)	NOT NULL	Total sum insured (INR)
RemainingCoverageAmount	DECIMAL(18,2)	NOT NULL	Coverage left after claims
TotalApprovedClaimsAmount	DECIMAL(18,2)	DEFAULT 0	Sum of all approved claim payouts
PaymentMode	NVARCHAR(MAX)	NULL	Annual/Semi-Annual/Quarterly/Monthly
NextPaymentDate	DATETIME2	NULL	Date of next premium payment
StartDate	DATETIME2	NULL	Policy start date (after payment)
ExpiryDate	DATETIME2	NULL	Policy expiry date
PaidAmount	DECIMAL(18,2)	NULL	Amount paid by customer
PaymentDate	DATETIME2	NULL	Date of payment
TransactionId	NVARCHAR(MAX)	NULL	Payment transaction reference

7.10 InsuranceClaims

Stores all insurance claims raised by customers against their active policies. Tracks the full claims lifecycle from submission to officer decision.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique claim ID
PolicyApplicationId	NVARCHAR(450)	FK -> PolicyApplications(Id) NO ACTION	Associated policy
UserId	NVARCHAR(450)	FK -> AspNetUsers(Id) CASCADE	Customer who raised the claim
RequestedAmount	DECIMAL(18,2)	NOT NULL	Amount requested by customer (INR)
ApprovedAmount	DECIMAL(18,2)	DEFAULT 0	Amount approved by Claims Officer (INR)
Description	NVARCHAR(MAX)	NOT NULL	Detailed incident description
IncidentDate	DATETIME2	NOT NULL	Date of the incident (ISO 8601)
SubmissionDate	DATETIME2	DEFAULT GETUTCDATE()	Date claim was submitted
IncidentType	NVARCHAR(MAX)	NOT NULL	Accident/Sickness/Death/Property Damage

Column	Data Type	Constraints	Description
IncidentLocation	NVARCHAR(MAX)	NOT NULL	Location where incident occurred
HospitalName	NVARCHAR(MAX)	NOT NULL	Name of treating hospital/clinic
HospitalizationRequired	BIT	NOT NULL	Whether inpatient admission was required
AffectedMemberName	NVARCHAR(MAX)	NULL	Name of affected family member (Family policies)
AffectedMemberRelation	NVARCHAR(MAX)	NULL	Relation: Spouse/Son/Daughter etc.
Status	NVARCHAR(MAX)	DEFAULT 'Pending'	Pending/Approved/Rejected
Remarks	NVARCHAR(MAX)	NULL	Officer notes or justification
AssignedClaimOfficerId	NVARCHAR(450)	FK -> AspNetUsers(Id) NO ACTION	Officer assigned by Admin
ApprovedByOfficerId	NVARCHAR(MAX)	NULL	ID of officer who made the decision
ProcessedAt	DATETIME2	NULL	Timestamp of officer decision

7.11 ClaimDocuments

Stores references to uploaded claim documents. Files are stored in cloud storage (ImageKit); this table holds their metadata and access URLs.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique document record ID
ClaimId	NVARCHAR(450)	FK -> InsuranceClaims(Id) CASCADE	Associated claim
FileId	NVARCHAR(MAX)	NOT NULL	Cloud storage file identifier
FileName	NVARCHAR(MAX)	NOT NULL	Original file name (e.g., medical_bill.pdf)
FileUrl	NVARCHAR(MAX)	NOT NULL	Public URL to access the file from cloud storage
FileSize	BIGINT	NOT NULL	File size in bytes
UploadedAt	DATETIME2	DEFAULT GETUTCDATE()	Upload timestamp (UTC)

7.12 Chats

Stores chat session metadata for conversations between customers and their assigned agents, linked to a specific policy.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique chat session ID
PolicyId	NVARCHAR(450)	FK -> PolicyApplications(Id) CASCADE	Associated policy application
CustomerId	NVARCHAR(MAX)	NOT NULL	Customer user ID
AgentId	NVARCHAR(MAX)	NOT NULL	Agent user ID
CustomerEmail	NVARCHAR(MAX)	NULL	Customer email (denormalized for display)
AgentEmail	NVARCHAR(MAX)	NULL	Agent email (denormalized for display)
PolicyName	NVARCHAR(MAX)	NULL	Policy category name
Category	NVARCHAR(MAX)	NULL	Insurance category (e.g., Health)
CoverageAmount	DECIMAL(18,2)	NOT NULL	Policy coverage amount
DateActivated	DATETIME2	NOT NULL	Date the policy was activated
LastMessage	NVARCHAR(MAX)	NULL	Preview of the most recent message
LastMessageTime	DATETIME2	NULL	Timestamp of last message
CreatedAt	DATETIME2	DEFAULT GETUTCDATE()	Chat session creation timestamp
UpdatedAt	DATETIME2	DEFAULT GETUTCDATE()	Last update timestamp

7.13 ChatMessages

Stores individual messages within a chat session, supporting real-time WhatsApp-style communication.

Column	Data Type	Constraints	Description
Id	NVARCHAR(450)	PRIMARY KEY	Unique message ID
ChatId	NVARCHAR(450)	FK -> Chats(Id) CASCADE	Associated chat session
SenderId	NVARCHAR(MAX)	NOT NULL	User ID of the message sender
SenderRole	NVARCHAR(MAX)	NOT NULL	Role of sender: Customer or Agent
Message	NVARCHAR(MAX)	NOT NULL	Text content of the message
Timestamp	DATETIME2	DEFAULT GETUTCDATE()	Message sent timestamp (UTC)

Column	Data Type	Constraints	Description
IsRead	BIT	DEFAULT 0	Read receipt flag

7.14 Database Relationship Summary

Table	Foreign Keys	Delete Rule
AspNetUserRoles	UserId -> AspNetUsers, RoleId -> AspNetRoles	CASCADE
AspNetUserClaims	UserId -> AspNetUsers	CASCADE
AspNetUserLogins	UserId -> AspNetUsers	CASCADE
AspNetUserTokens	UserId -> AspNetUsers	CASCADE
AspNetRoleClaims	RoleId -> AspNetRoles	CASCADE
Notifications	UserId -> AspNetUsers	CASCADE
PolicyApplications	UserId -> AspNetUsers (CASCADE), AssignedAgentId -> AspNetUsers (NO ACTION)	Mixed
InsuranceClaims	PolicyApplicationId -> PolicyApplications (NO ACTION), UserId -> AspNetUsers (CASCADE), AssignedClaimOfficerId -> AspNetUsers (NO ACTION)	Mixed
ClaimDocuments	ClaimId -> InsuranceClaims	CASCADE
Chats	PolicyId -> PolicyApplications	CASCADE
ChatMessages	ChatId -> Chats	CASCADE

8. API Controllers and Endpoints

The AcciSure REST API is hosted at https://api.insuranceplatform.com and consists of 10 controllers with 44 endpoints. All protected endpoints require a JWT Bearer token in the Authorization header. The API uses application/json for all requests except file uploads, which use multipart/form-data. All monetary values are in INR and all datetimes use ISO 8601 UTC format.

8.1 AuthController — /api/auth

Handles customer registration and login. No authentication required for either endpoint.

#	Method	Endpoint	Auth	Description
1.1	POST	/api/auth/register	None	Register new customer. Required fields: fullName, email, password (min 8 chars, mixed case + digit + symbol), phoneNumber. Optional: aadhaarNumber. Returns JWT, email, roles, userId.
1.2	POST	/api/auth/login	None	Authenticate user. Required: email, password. Returns JWT Bearer token, email, roles, userId.

8.2 AdminController — /api/admin

Provides full administrative control. All 11 endpoints require Admin role.

#	Method	Endpoint	Description
2.1	POST	/api/admin/create-agent	Create agent account. Fields: fullName, email, phoneNumber, bankAccountNumber (16-digit), password.
2.2	POST	/api/admin/create-claim-officer	Create claims officer account. Same fields as create-agent. Returns userId and email.
2.3	GET	/api/admin/agents	List all agent accounts in the system.
2.4	GET	/api/admin/claim-officers	List all claims officer accounts.
2.5	DELETE	/api/admin/delete-user/{id}	Permanently delete any user by their ID.
2.6	GET	/api/admin/policy-requests	View all policy applications across all statuses.
2.7	GET	/api/admin/agents-with-load	List agents with their current assigned policy count for load-based assignment.
2.8	POST	/api/admin/assign-agent	Assign a policy application to an agent. Body: applicationId, agentId.
2.9	GET	/api/admin/all-users	List all platform users: customers, agents, and officers.
2.10	GET	/api/admin/all-claims	Master view of all insurance claims across the platform.

#	Method	Endpoint	Description
2.11	GET	/api/admin/admin-stats	KPI dashboard statistics: user counts, policy counts by status, claim counts by status, total and approved claim amounts.

8.3 AgentController — /api/agent

Agent-specific operations for reviewing applications, managing customers, and tracking performance. All endpoints require Agent role.

#	Method	Endpoint	Description
3.1	GET	/api/agent/my-requests	Fetch all policy applications assigned to the logged-in agent.
3.2	POST	/api/agent/review-request	Approve or reject a policy application. Body: applicationId, status ('Approved' or 'Rejected').
3.3	GET	/api/agent/my-customers	List all customers with active policies under this agent.
3.4	GET	/api/agent/commission-stats	Total commission earned and per-policy commission breakdown.
3.5	GET	/api/agent/analytics	Full analytics: totalCoverageProvided, activePolicyCount, uniqueCustomerCount, bestPerformingCategory, bestPerformingTier, totalPremiumCollected, totalCommissionEarned, commissionPerformance (monthly array), portfolioMix, tierBreakdown.

8.4 PolicyController — /api/policy

Manages policy configuration, premium calculations, applications, and customer policy views.

#	Method	Endpoint	Auth	Description
4.1	GET	/api/policy/configuration	None (Public)	Returns all policy plans with tiers (GOLD/SILVER), pricing, benefits, and risk factor multipliers. Age multipliers: 18-30 = 1.0x, 31-45 = 1.2x, 46-60 = 1.5x.
4.2	POST	/api/policy/calculate-premium	JWT	Calculate premium quote without submitting. Fields: policyCategory, tierId, applicant (fullName, age 18-60, profession, alcoholHabit, smokingHabit, travelKmPerMonth), paymentMode, nominee details, members array (Family only).
4.3	POST	/api/policy/apply	JWT	Submit formal policy application. Accepts same body as calculate-premium. Returns applicationId, calculatedPremium, status: Pending.
4.4	GET	/api/policy/my-policies	JWT	Returns all policies for authenticated customer with fields: id, policyCategory,

#	Method	Endpoint	Auth	Description
				tierId, status, calculatedPremium, totalCoverageAmount, startDate, expiryDate, assignedAgentId.

8.5 ClaimController — /api/claim

Manages the full insurance claim lifecycle from filing through officer decision. Access is role-based per endpoint.

#	Method	Endpoint	Role	Description
5.1	POST	/api/claim/raise	Customer	File a new claim (multipart/form-data). Fields: policyApplicationId, incidentType, incidentLocation, incidentDate, description, hospitalName, hospitalizationRequired, requestedAmount, affectedMemberName, affectedMemberRelation, documents (PDF/JPG/PNG, max 10 files, 5 MB each).
5.2	GET	/api/claim/my-claims	Customer	View all claims raised by the authenticated customer.
5.3	GET	/api/claim/admin/pending	Admin	List all pending claims awaiting officer assignment.
5.4	GET	/api/claim/admin/officers	Admin	Officer workload — current claim count per officer.
5.5	POST	/api/claim/admin/assign	Admin	Assign a claims officer to a pending claim.
5.6	GET	/api/claim/officer/my-requests	ClaimOfficer	Get all claims assigned to the logged-in officer (claims queue).
5.7	POST	/api/claim/officer/review	ClaimOfficer	Approve or reject a claim. Body: claimId, status ('Approved'/'Rejected'), approvedAmount (INR; 0 if rejected), remarks.
5.8	GET	/api/claim/agent/customer-claims	Agent	View claims raised by the agent's customers.

8.6 ChatController — /api/chat

Enables real-time messaging between customers and agents. JWT required for all endpoints.

#	Method	Endpoint	Description
6.1	GET	/api/chat/list	Get all chat conversations for the authenticated user (customer sees their chats; agent sees all assigned policy chats).
6.2	GET	/api/chat/{policyId}	Retrieve full message history for a specific policy's chat session.
6.3	POST	/api/chat/init	Initialize a new chat session for a policy application. Body: policyId.
6.4	POST	/api/chat/{policyId}/read	Mark all messages in a policy chat as read (read receipts).

8.7 NotificationsController — /api/notifications

Manages user notifications triggered by policy status changes, claim updates, and system events. JWT required.

#	Method	Endpoint	Description
7.1	GET	/api/notifications	Get all notifications for the authenticated user.
7.2	GET	/api/notifications/unread-count	Returns integer count of unread notifications.
7.3	POST	/api/notifications/{id}/read	Mark a specific notification as read by its ID.
7.4	POST	/api/notifications/read-all	Mark all notifications as read in one call.

8.8 PaymentController — /api/payment

Processes premium payments for approved policy applications. JWT required.

#	Method	Endpoint	Description
8.1	POST	/api/payment/process	Process premium payment. Body: applicationId (approved policy), amount (must match calculatedPremium in INR). Returns: paymentId, policyStatus: Active.

8.9 ReportController — /api/report

Provides role-filtered financial reports. Accessible by Admin, Agent, and ClaimOfficer.

#	Method	Endpoint	Description
9.1	GET	/api/report/unified-payments	Role-filtered report: Admin sees all payments/payouts; Agent sees their portfolio payments; ClaimOfficer sees claim payout transactions they handled.

8.10 DashboardController — /api/dashboard

Returns 200 OK if the caller has the correct role, 403 Forbidden otherwise. Used for frontend access validation and role-gating.

#	Method	Endpoint	Required Role
10.1	GET	/api/dashboard/customer	Customer
10.2	GET	/api/dashboard/agent	Agent
10.3	GET	/api/dashboard/claim-officer	ClaimOfficer
10.4	GET	/api/dashboard/admin	Admin

8.11 Error Handling — Standard Response Format

All errors follow a consistent JSON format: { status, message, errors[] }

HTTP Code	Status	Scenario
200	OK	Request successful
201	Created	Resource created successfully
400	Bad Request	Validation failed or business rule violation (e.g., 'Email already exists', 'Age must be between 18 and 60')
401	Unauthorized	Missing or expired JWT token
403	Forbidden	Authenticated but wrong role for this endpoint
404	Not Found	Requested resource does not exist
429	Too Many Requests	Rate limit exceeded (100 requests/minute/user)
500	Internal Server Error	Unhandled server-side error

9. System Requirements

9.1 Frontend Development Requirements

Category	Technology / Requirement
Framework	React.js (with Vite or Next.js) or Angular — component-based SPA architecture
Styling	Tailwind CSS or Material UI for responsive, modern UI design
State Management	Redux Toolkit or React Context API for global state (auth, notifications)
HTTP Client	Axios for REST API communication with interceptors for JWT token attachment
Real-Time Chat	SignalR (Microsoft) or Socket.IO client for WebSocket-based messaging
Video Calling	WebRTC integration (e.g., Agora SDK, Jitsi, or Daily.co) for in-platform video calls
AI Chatbot	Embedded iframe or webhook-based integration with N8N chatbot workflow
OTP Verification	Phone OTP integration via Twilio or similar SMS gateway
CAPTCHA	Google reCAPTCHA v2 or v3 for registration security
File Upload	Multipart form data handling for claim document uploads (PDF, JPG, PNG)
Authentication	JWT token storage (HttpOnly cookies recommended), auto-refresh logic, role-based route guards
Forms	React Hook Form or Formik with Yup validation for complex policy application forms
Notifications	Toast notifications (e.g., react-hot-toast) and unread badge count on header
Charts	Recharts or Chart.js for agent analytics dashboards and admin KPI visualizations
Routing	React Router v6 with protected routes gated by JWT roles
Browser Support	Chrome, Firefox, Edge (latest two versions); mobile-responsive design

9.2 Backend Development Requirements

Category	Technology / Requirement
Framework	ASP.NET Core 8.0 Web API — RESTful architecture with controller-based routing
Language	C# with nullable reference types and async/await patterns throughout
Authentication	ASP.NET Core Identity + JWT Bearer token authentication (Microsoft.AspNetCore.Authentication.JwtBearer)
Authorization	[Authorize(Roles = 'Admin')] attribute-based role authorization per endpoint
ORM	Entity Framework Core 8 with SQL Server provider (Microsoft.EntityFrameworkCore.SqlServer)
Database	Microsoft SQL Server 2019+ (SQL Server LocalDB for development)
Real-Time	ASP.NET Core SignalR for WebSocket-based real-time chat functionality
File Storage	ImageKit.io SDK for cloud document upload and URL generation
Email	MailKit or SendGrid SDK for automated credential delivery emails
AI Integration	N8N webhook calls for AI chatbot and claims analysis workflows
Rate Limiting	100 requests per minute per user/IP (ASP.NET Core rate limiting middleware)
CORS	Configured allowed origins for browser-based clients (React frontend domain)
Logging	Serilog or built-in ILogger with structured logging to file and console
API Documentation	Swagger/OpenAPI auto-generated documentation via Swashbuckle
Data Validation	FluentValidation or Data Annotations for request model validation
Dependency Injection	Built-in ASP.NET Core DI container — all services registered in Program.cs

10. System Architecture

AcciSure follows a clean three-tier architecture separating the presentation layer, business logic, and data persistence.

10.1 Architecture Overview

Layer	Technology	Responsibility
Presentation Layer	React.js / Angular SPA	User interface, role-based routing, real-time chat UI, policy browsing, AI chatbot integration, video calling, form handling, notification display
API Layer	ASP.NET Core 8 Web API	RESTful endpoints (44 endpoints across 10 controllers), JWT auth, role authorization, request validation, business logic orchestration, SignalR hub for chat
Data Layer	SQL Server + EF Core	Relational data persistence with 13 tables, foreign key constraints, indexes for performance, LINQ-based queries
Cloud Storage	ImageKit.io	Claim document upload and CDN delivery. Files uploaded on claim submission; URLs stored in ClaimDocuments table
AI/Automation	N8N Workflow Engine	AI chatbot for policy queries; AI claims analysis (summary, risk score, payout range, risk rating) triggered on claims review
Notifications	Email Automation (SMTP/SendGrid)	Automated credential delivery to agents and claims officers; system notifications via email
Auth/Security	JWT Bearer + ASP.NET Identity	Token-based stateless authentication; bcrypt password hashing; role-based access control; OTP + CAPTCHA at registration

10.2 Data Flow Architecture

Client sends HTTP request with JWT Bearer token. API Gateway validates token and routes to appropriate controller. Controller invokes service layer which interacts with Entity Framework Core. EF Core executes parameterized SQL queries against SQL Server. For file operations, service calls ImageKit SDK. For AI features, service calls N8N webhook. Response is serialized to JSON and returned to client. For real-time events, SignalR hub broadcasts messages to connected clients.

10.3 Security Architecture

- Authentication: JWT Bearer tokens issued on login, validated on every protected request.
- Password Security: ASP.NET Identity handles bcrypt hashing with salting.
- Registration Security: OTP verification + CAPTCHA prevents bot registrations.
- Authorization: Role-based access control using [Authorize(Roles)] attributes on all sensitive endpoints.
- Rate Limiting: 100 requests/minute/user enforced at API level to prevent abuse.
- File Validation: Only PDF, JPG, JPEG, PNG accepted; max 5 MB per file; max 10 files per claim.
- CORS: Configured allowed origins restrict browser access to trusted frontend domains.

11. Workflow Summaries — Login to Logout

11.1 Customer Workflow

Step	Action	Details
1	Registration	Customer navigates to Register page. Enters fullName, email, password, phoneNumber, aadhaarNumber. Completes phone OTP verification and CAPTCHA. POST /api/auth/register returns JWT token.
2	Login	POST /api/auth/login with email and password. JWT token stored. GET /api/dashboard/customer validates role access.
3	Policy Exploration	Customer browses GET /api/policy/configuration to view all plans, tiers, pricing, and benefits.
4	AI Chatbot	Customer clicks 'Interest to Buy' on a policy. N8N-powered chatbot sidebar opens. Customer asks questions; AI responds in real time. Customer closes chatbot when satisfied.
5	Premium Calculation	Customer fills applicant details. POST /api/policy/calculate-premium returns a quote without submitting.
6	Policy Application	Customer clicks 'Buy Now', fills nominee details, selects payment mode. POST /api/policy/apply creates application with status 'Pending'. Notification sent.
7	Await Assignment & Review	Admin assigns agent via POST /api/admin/assign-agent. Agent reviews and approves via POST /api/agent/review-request. Customer receives notification of approval.
8	Premium Payment	Customer sees approved policy. POST /api/payment/process with applicationId and amount. Policy status changes to 'Active'. Agent receives 10% commission.
9	Chat with Agent	Customer opens Chat section. GET /api/chat/list shows conversations. POST /api/chat/init to start. Real-time messaging via SignalR. Customer can also initiate video call.
10	Raise a Claim	Customer clicks active policy, selects 'Raise a Claim'. Fills incident details, uploads documents. POST /api/claim/raise (multipart). Documents go to ImageKit cloud; URLs stored in DB.
11	Claim Progress	Admin assigns claims officer. Officer reviews and decides. Customer receives notification. GET /api/claim/my-claims shows updated status and approved amount.
12	Logout	JWT token cleared from client storage. Session ends.

11.2 Agent Workflow

Step	Action	Details
1	Login	Agent receives credentials via email from Admin. POST /api/auth/login. GET /api/dashboard/agent validates role.
2	View Assigned Applications	GET /api/agent/my-requests to see policy applications assigned by Admin.
3	Review Application	Agent reads full ApplicationDataJson (applicant info, nominee, members). Decides to accept or reject.
4	Submit Decision	POST /api/agent/review-request with applicationId and status ('Approved' or 'Rejected'). Customer is notified.
5	Customer Communication	GET /api/chat/list to view customer chats. Real-time messaging and video calls with assigned customers.
6	Monitor Portfolio	GET /api/agent/my-customers for active customer list. GET /api/agent/commission-stats for earnings. GET /api/agent/analytics for full performance analytics.
7	View Customer Claims	GET /api/claim/agent/customer-claims to see claims raised by customers in their portfolio.
8	Financial Reports	GET /api/report/unified-payments for payment data related to their portfolio.
9	Logout	JWT token cleared. Session ends.

11.3 Claims Officer Workflow

Step	Action	Details
1	Login	Claims Officer receives credentials from Admin via email. POST /api/auth/login. GET /api/dashboard/claim-officer validates role.
2	View Claims Queue	GET /api/claim/officer/my-requests retrieves all assigned claims.
3	Select Claim	Officer clicks 'View Details' on a claim in the queue. Full incident details, hospital info, and uploaded document URLs are displayed.
4	AI-Assisted Review	N8N AI workflow is triggered automatically, analyzing all attached documents and returning: Summary, Risk Factor, Claim Amount Range, Risk Rating.
5	Make Decision	Officer reviews AI analysis alongside documents and incident details. POST /api/claim/officer/review with claimId, status, approvedAmount, remarks.
6	Post-Decision	If approved, InsuranceClaims.ApprovedAmount and ProcessedAt are updated. PolicyApplications.RemainingCoverageAmount is reduced accordingly. Customer is notified.
7	Financial Reports	GET /api/report/unified-payments for claim payout transactions handled by this officer.
8	Logout	JWT token cleared. Session ends.

11.4 Admin Workflow

Step	Action	Details
1	Login	Admin logs in with pre-configured credentials. POST /api/auth/login. GET /api/dashboard/admin validates role.
2	Dashboard Overview	GET /api/admin/admin-stats for platform-wide KPIs: user counts, policy statuses, claim counts and amounts.
3	Create Staff Accounts	POST /api/admin/create-agent or /create-claim-officer with full details. System stores InitialPassword. Email automation dispatches credentials.
4	Manage Users	GET /api/admin/all-users for full user list. GET /api/admin/agents and /claim-officers for filtered views. DELETE /api/admin/delete-user/{id} to remove users.
5	Assign Policy Applications	GET /api/admin/policy-requests to see new applications. GET /api/admin/agents-with-load to check workload. POST /api/admin/assign-agent with applicationId and agentId.
6	Assign Claims	GET /api/admin/all-claims for all claims. GET /api/claim/admin/pending for unassigned claims. GET /api/claim/admin/officers for officer workload. POST /api/claim/admin/assign to assign officer.
7	Monitor Platform	GET /api/admin/admin-stats regularly for KPI tracking. GET /api/report/unified-payments for complete financial oversight.
8	Logout	JWT token cleared. Session ends.

12. Unique Selling Propositions (USPs)

AcciSure differentiates itself from conventional insurance platforms through the following key USPs:

USP	Description	Competitive Advantage
In-Platform Video Calling	Customers and agents can initiate HD video calls directly within the platform without switching to third-party apps.	No other accidental insurance platform in the Indian market offers native in-platform video consultation — eliminating the need for Zoom or Google Meet links.
AI Policy Chatbot (Pre-Purchase)	An N8N-powered AI chatbot opens inline when a customer shows interest in a policy, answering specific questions about coverage, exclusions, and premiums before commitment.	Reduces policy misunderstandings and returns, increases conversion rates, and creates a self-service experience at scale.
AI Claims Intelligence	Every claim reviewed by an officer comes pre-analyzed by AI: document summary, risk factor, suggested payout range, and risk rating — reducing review time from hours to minutes.	Faster claim settlement, reduced fraud risk, and improved officer decision accuracy through data-driven insights.
WhatsApp-Style Chat	Real-time, persistent messaging between customers and their dedicated agent, with full message history and read receipts — all within the insurance platform.	Creates a trusted relationship channel and reduces customer support call volume significantly.
Smart Load-Based Assignment	Admin sees real-time workload metrics per agent and officer, enabling evidence-based assignment to prevent bottlenecks.	Eliminates idle staff, prevents burnout, and ensures consistent service quality across the customer base.
Live Coverage Balance Tracking	The remaining coverage balance is updated in real time after every approved claim, giving customers full transparency into their current coverage position.	Builds trust and reduces disputes by maintaining a clear, always-current ledger of policy utilization.
Automated Commission Management	Agent commissions (10% of premium) are auto-calculated and recorded on every policy activation, providing a verifiable commission ledger.	Eliminates manual commission calculations and disputes, incentivizing agents through transparent, automated payouts.

13. Conclusion

AcciSure represents a significant leap forward in the digitization of accidental insurance services. By bringing together a robust multi-role architecture, AI-powered assistance at every critical touchpoint, real-time communication features, and automated workflow management, AcciSure addresses the fundamental pain points that have long plagued the traditional insurance industry.

The platform's four-role ecosystem — Customer, Agent, Claims Officer, and Admin — operates through a carefully designed, end-to-end workflow backed by 44 REST API endpoints, a 13-table relational SQL Server database, and deep integrations with cloud storage, AI automation, and real-time communication technologies.

What truly distinguishes AcciSure is not any single feature, but the seamless integration of all components into a unified experience. A customer can discover a policy, get all their questions answered by AI, apply and pay within minutes, communicate with their dedicated agent in real time via chat or video call, and receive their claim settlement with full transparency — all without leaving the platform.

The AI integrations through N8N workflow automation — both for pre-purchase guidance and post-submission claims analysis — position AcciSure not just as a software application, but as an intelligent insurance partner that augments human decision-making with data-driven insights.

As the insurance industry in India continues its digital transformation, AcciSure's architecture is designed to scale: its JWT-based stateless API, cloud document storage, and modular controller design allow for seamless expansion of features, policies, and user volumes without architectural rework.

In summary, AcciSure is a production-ready, comprehensive insurance management platform that demonstrates how thoughtful software engineering, AI integration, and user-centric design can together transform a traditionally opaque and slow industry into one that is transparent, fast, and genuinely helpful to all stakeholders — customers, agents, officers, and administrators alike.